

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE



**INTELIGENCIA ARTIFICIAL
SECCIÓN 2**

Búsqueda Informada y No Informada

Aplicación a la búsqueda de rutas en un mapa de ciudades del Perú

GRUPO 06

INTEGRANTES:

Bautista De la Cruz Claudia Daniela

Carrascal Castro Priscila Maria

Ccahuana Quiñones Judith Valeria

Gil Sixi Alberto Luis

Medrano Ayma Nikol Arlet

Rosales Trinidad Jeanmarco Miguel

Saire Tello Fernando José

DOCENTE

Prof. Juan Gamarra Moreno

Lima, Perú – 2026

ÍNDICE

1. Introducción	3
2. Descripción y modelado del problema	3
2.1. Definición del problema	3
2.2. Espacio de estados	4
2.3. Operadores y costos	5
3. Diseño de la solución y algoritmos	6
3.1. Búsqueda no informada BFS	6
3.2. Búsqueda no informada DFS	6
3.3. Búsqueda informada Greedy Best-First	7
3.4. Búsqueda informada A*	7
3.5. Heurística empleada	8
4. Implementación	10
4.1. Estructura del código	10
4.2. Fragmentos comentados	11
5. Resultados y análisis comparativo	14
5.1. Casos de prueba	14
5.2. Tabla comparativa	16
5.3. Análisis de resultados	17
6. Conclusiones	18
7. Anexo de prompts	19
8. Referencias	20

1. Introducción

El presente informe aborda un problema clásico de la IA: la búsqueda de rutas óptimas dentro de un grafo que modela un conjunto de ciudades interconectadas del Perú. El objetivo central es encontrar el camino de menor costo (medido en kilómetros de carretera) entre una ciudad de origen y una ciudad de destino, aplicando cuatro algoritmos de búsqueda distintos.

Para cumplir este objetivo se implementaron, en lenguaje Java, dos algoritmos de búsqueda no informada: **Búsqueda en Anchura (BFS)** y **Búsqueda en Profundidad (DFS)**. Asimismo, se implementaron ambos algoritmos de búsqueda informada: **Greedy Best-First Search** y **A***. Estos algoritmos representan enfoques fundamentalmente distintos: los no informados exploran el espacio de estados sin conocimiento adicional sobre el destino, mientras que los informados utilizan una función heurística para guiar la búsqueda de manera más eficiente hacia la meta.

El informe documenta el modelado formal del problema, el diseño e implementación de cada algoritmo, los resultados obtenidos en distintos casos de prueba y un análisis comparativo que permite evaluar el desempeño de cada enfoque en términos de optimalidad, completitud, nodos expandidos y tiempo de ejecución.

2. Descripción y modelado del problema

2.1. Definición del problema

El problema consiste en encontrar la ruta de menor distancia total entre dos ciudades del Perú, recorriendo únicamente las carreteras existentes que las conectan. Dado que una persona desea viajar desde una ciudad de origen hasta una ciudad de destino, se busca cuál es la secuencia de ciudades intermedias que minimiza la distancia total recorrida en kilómetros.

Para representar este escenario de forma computacionalmente tratable, el problema se modela como un **grafo no dirigido y ponderado**, en el que cada nodo representa una ciudad peruana y cada arista representa una carretera que conecta dos ciudades, con un peso numérico que expresa la distancia

aproximada en kilómetros entre ellas. La naturaleza no dirigida del grafo refleja que las carreteras son transitables en ambas direcciones.

El mapa implementado incluye las siguientes doce ciudades: **Lima, Trujillo, Arequipa, Cusco, Piura, Tacna, Ica, Puno, Chiclayo, Huancayo, Cajamarca y Pucallpa**. Las conexiones y distancias entre ellas fueron definidas con base en las rutas terrestres reales y se detallan en la sección 2.3.

La comparación entre los cuatro algoritmos implementados permite evaluar cuándo conviene invertir en una heurística (búsqueda informada) frente a explorar sin guía adicional (búsqueda no informada). Como se retomará en la sección 3.5, la distancia en línea recta entre coordenadas geográficas sirve como heurística para los algoritmos informados, ya que ofrece una estimación del costo restante sin sobreestimar la distancia real por carretera.

2.2. Espacio de estados

El espacio de estados del problema se define formalmente de la siguiente manera:

- **Estado:** Un estado es una ciudad del mapa en la que se encuentra actualmente el agente viajero. Cada estado queda identificado por el nombre de la ciudad y el costo acumulado desde el origen hasta dicha ciudad. Formalmente, un estado es un par $(ciudad, g)$, donde g es el costo de camino recorrido hasta el momento.
- **Estado inicial:** La ciudad de origen que el usuario ingresa al ejecutar el programa. Por ejemplo, $(Lima, 0)$.
- **Estado(s) meta:** La ciudad de destino ingresada por el usuario. El agente alcanza la meta cuando el nodo en expansión corresponde a dicha ciudad, sin importar el camino seguido para llegar a ella. Por ejemplo, $(Cusco, g)$ para cualquier valor de g .
- **Espacio de búsqueda:** El conjunto de todos los estados posibles está formado por las doce ciudades del mapa. El espacio es finito y, dado

que el grafo es conexo, siempre existe al menos una ruta entre cualquier par de ciudades.

2.3. Operadores y costos

Los operadores representan las acciones disponibles para el agente en cada estado: desplazarse desde la ciudad actual hacia cualquier ciudad vecina conectada por una carretera directa. Cada operador tiene un **costo** igual a la distancia en kilómetros de esa carretera, que es el valor de la arista en el grafo.

La tabla siguiente lista todas las conexiones del mapa con sus costos asociados. Dado que el grafo es no dirigido, cada par puede recorrerse en ambas direcciones con el mismo costo.

Ciudad A	Ciudad B	Distancia (km)
Lima	Trujillo	557
Lima	Ica	303
Lima	Huancayo	298
Trujillo	Chiclayo	209
Trujillo	Cajamarca	295
Chiclayo	Piura	241
Cajamarca	Piura	448
Cajamarca	Pucallpa	940
Ica	Arequipa	718
Huancayo	Pucallpa	730
Huancayo	Cusco	736
Arequipa	Cusco	521
Arequipa	Tacna	319
Arequipa	Puno	297
Cusco	Puno	388
Cusco	Pucallpa	833
Puno	Tacna	377

El agente selecciona en cada paso uno de los operadores disponibles desde su ciudad actual. La secuencia de operadores aplicados desde el estado inicial hasta el estado meta define la ruta encontrada, y la suma de los costos de esos operadores es el costo total de la solución.

3. Diseño de la solución y algoritmos

3.1. Búsqueda no informada BFS

La **Búsqueda en Anchura** (Breadth-First Search) explora el grafo nivel por nivel: primero expande todos los nodos a profundidad 1 desde el origen, luego los de profundidad 2, y así sucesivamente. No utiliza ninguna información sobre la posición del destino; simplemente garantiza que el primer camino encontrado hacia la meta sea el de menor número de pasos (menor número de aristas).

En la implementación, BFS utiliza una **cola FIFO** (First In, First Out) como estructura de frontera. Cada elemento de la cola es un nodo que almacena: la ciudad actual, el costo acumulado **g**, el camino recorrido y el conjunto de ciudades ya visitadas para evitar ciclos. Al desencolar un nodo, se generan todos sus vecinos no visitados y se agregan al final de la cola.

Dado que el grafo tiene pesos (distancias en km), BFS no garantiza encontrar la ruta de menor **costo**, sino la de menor número de **ciudades intermedias**. Esta distinción es importante para interpretar los resultados de la tabla comparativa. (Russell & Norvig, 2022).

3.2. Búsqueda no informada DFS

La **Búsqueda en Profundidad** (Depth-First Search) explora el grafo siguiendo un camino hasta que llega a un callejón sin salida o encuentra la meta, y solo entonces retrocede para explorar alternativas. Tampoco emplea información sobre el destino.

La implementación utiliza una **pila LIFO** (Last In, First Out) como estructura de frontera, lo que produce el comportamiento característico de profundización. Al igual que en BFS, cada nodo almacena la ciudad actual, el

costo acumulado, el camino recorrido y el conjunto de visitados para evitar ciclos. Los vecinos de cada nodo se apilan en orden inverso para mantener consistencia con el orden de exploración del grafo.

DFS es **completo** en grafos finitos con detección de ciclos, pero **no es óptimo**: puede encontrar una solución con costo elevado porque no considera los pesos de las aristas al decidir qué camino explorar primero. (Russell & Norvig, 2022).

3.3. Búsqueda informada Greedy Best-First

La **Búsqueda Greedy Best-First** utiliza una función heurística $h(n)$ para ordenar la frontera de búsqueda, expandiendo siempre el nodo que parece más cercano al destino según dicha estimación. A diferencia de BFS y DFS, este algoritmo sí tiene conocimiento sobre la posición del destino, lo que le permite orientar la búsqueda de forma más eficiente.

La implementación usa una **cola de prioridad** (min-heap) donde la prioridad de cada nodo está dada exclusivamente por $h(n)$: la distancia en línea recta desde la ciudad actual hasta la ciudad destino, calculada a partir de sus coordenadas geográficas (latitud y longitud). El nodo con menor $h(n)$ se expande primero.

Greedy Best-First es generalmente **más rápido** que BFS y DFS porque expande pocos nodos, pero **no garantiza optimalidad**: puede elegir caminos que parecen prometedores localmente pero que resulten más costosos en total. (Russell & Norvig, 2022).

3.4. Búsqueda informada A*

El algoritmo A* combina lo mejor de la búsqueda de costo uniforme (que garantiza optimalidad) y la búsqueda greedy (que es eficiente). Para cada nodo n calcula la función:

$$f(n) = g(n) + h(n)$$

donde $g(n)$ es el costo real acumulado desde el origen hasta n , y $h(n)$ es la estimación heurística del costo restante desde n hasta la meta. La frontera de búsqueda se implementa como una **cola de prioridad** ordenada por $f(n)$.

A* expande primero el nodo con menor $f(n)$, garantizando que cuando la meta es extraída de la cola, el camino encontrado es óptimo, **siempre que la heurística sea admisible** (no sobreestime el costo real). Además, si la heurística es también **consistente** (monotónica), A* nunca necesita reabrir nodos ya expandidos, lo que mejora su eficiencia.

La implementación mantiene dos conjuntos: la frontera (cola de prioridad) y el conjunto de nodos ya cerrados (expandidos), para evitar reexpansión innecesaria. (Russell & Norvig, 2022; Skiena, 2020).

3.5. Heurística empleada

La heurística utilizada tanto en Greedy Best-First como en A* es la **distancia en línea recta** entre la ciudad actual y la ciudad destino, calculada mediante la fórmula de Haversine sobre las coordenadas geográficas reales (latitud y longitud en grados decimales) de cada ciudad peruana.

La fórmula de Haversine calcula la distancia entre dos puntos sobre la superficie terrestre:

$$a = \sin^2(\Delta lat/2) + \cos(lat_1) \cdot \cos(lat_2) \cdot \sin^2(\Delta lon/2)$$

$$h(n) = 2 \cdot R \cdot \arcsin(\sqrt{a})$$

Donde:

- lat_1, lon_1 : latitud y longitud de la ciudad actual (en radianes)
- lat_2, lon_2 : latitud y longitud de la ciudad destino (en radianes)
- $\Delta lat = lat_2 - lat_1$ y $\Delta lon = lon_2 - lon_1$
- $R = 6371 \text{ km}$: radio medio de la Tierra

Las coordenadas geográficas de cada ciudad utilizadas en el programa son las siguientes (Instituto Geográfico Nacional del Perú, 2024):

Ciudad	Latitud	Longitud
Lima	-12.0464	-77.0428
Trujillo	-8.1120	-79.0288
Arequipa	-16.4090	-71.5375
Cusco	-13.5320	-71.9675
Piura	-5.1945	-80.6328
Tacna	-18.0060	-70.2481
Ica	-14.0678	-75.7286
Puno	-15.8402	-70.0219
Chiclayo	-6.7714	-79.8409
Huancayo	-12.0653	-75.2049
Cajamarca	-7.1620	-78.5127
Pucallpa	-8.3791	-74.5539

- Admisibilidad:** La heurística es admisible porque la distancia en línea recta entre dos puntos nunca puede superar la distancia real por carretera entre ellos. La carretera debe sortear accidentes geográficos (montañas, ríos, curvas), por lo que siempre es mayor o igual a la distancia en línea recta. En consecuencia, $h(n) \leq h^*(n)$ para todo nodo n , donde $h^*(n)$ es el costo real óptimo desde n hasta la meta. (Russell & Norvig, 2022).
- Consistencia:** La heurística es consistente porque satisface la desigualdad triangular: para todo nodo n y su sucesor n' mediante una acción de costo c , se cumple $h(n) \leq c(n, n') + h(n')$. Esto se desprende directamente de la desigualdad triangular geométrica

aplicada a distancias euclidianas sobre la superficie esférica. La consistencia garantiza que A* no necesite reabrir nodos y que su solución sea óptima.

4. Implementación

4.1. Estructura del código

El programa está compuesto por diez clases Java organizadas por responsabilidad única. La tabla siguiente describe el rol de cada una:

1. **Par.java** almacena un par ciudad–distancia y es la unidad básica con la que el grafo representa cada vecino: contiene un campo `String ciudad` y un campo `int distancia`.
2. **Resultado.java** encapsula la salida de cada algoritmo de búsqueda: la ruta como lista de ciudades (`List<String> ruta`), el costo total en km (`double costo`), el número de nodos expandidos (`int nodosExpandidos`) y el tiempo de ejecución en milisegundos (`long tiempoMs`).
3. **Grafo.java** representa el mapa como un grafo no dirigido y ponderado mediante una lista de adyacencia implementada con `LinkedHashMap<String, List<Par>>`. Expone tres métodos: `addArista()` para agregar conexiones bidireccionales, `getVecinos()` para obtener los vecinos de una ciudad y `getCiudades()` para listar todas las ciudades disponibles.
4. **Nodo.java** representa un estado de búsqueda dentro del árbol de exploración. Almacena la ciudad actual, el costo acumulado g , la estimación heurística h , la función de evaluación $f = g + h$, el camino recorrido desde el origen y el conjunto de ciudades ya visitadas en ese camino para evitar ciclos. Implementa `Comparable<Nodo>` para ser usado en colas de prioridad, comparando por f .
5. **Heuristica.java** contiene el método estático `distanciaLineaRecta(String ciudad1, String ciudad2)` que calcula la distancia en kilómetros entre dos ciudades aplicando la fórmula de Haversine sobre un mapa interno de coordenadas geográficas de las doce ciudades.
6. **BFS.java** implementa la Búsqueda en Anchura. Su método `buscar()` devuelve un objeto `Resultado` con la ruta, costo, nodos expandidos y tiempo de ejecución.

7. **DFS.java** implementa la Búsqueda en Profundidad con la misma interfaz que **BFS**.
8. **Greedy.java** implementa Greedy Best-First con una cola de prioridad ordenada por $h(n)$.
9. **AStar.java** implementa A* con una cola de prioridad ordenada por $f(n) = g(n) + h(n)$ y un conjunto de nodos cerrados para evitar reexpansiones.
10. **BusquedaRutasPeru.java** es la clase principal. Su método `main()` construye el grafo con las 12 ciudades y 17 conexiones, solicita al usuario la ciudad de origen y la de destino por consola, instancia y ejecuta los cuatro algoritmos en secuencia, imprime los resultados individuales de cada uno y muestra al final la tabla comparativa consolidada.

El programa se ejecuta por consola sin interfaz gráfica, el entorno de desarrollo utilizado es Java SE 17 (Oracle Corporation, 2024). Los comandos de compilación y ejecución son los siguientes:

```
javac *.java
java BusquedaRutasPeru
```

4.2. Fragmentos comentados

```
// Se crea el grafo y se agregan todas las aristas del mapa peruano.
// addArista() registra la conexión en ambas direcciones (grafo no dirigido).
Grafo mapa = new Grafo();
mapa.addArista("Lima", "Trujillo", 557);
mapa.addArista("Lima", "Ica", 303);
mapa.addArista("Lima", "Huancayo", 298);
mapa.addArista("Trujillo", "Chiclayo", 209);
mapa.addArista("Trujillo", "Cajamarca", 295);
mapa.addArista("Chiclayo", "Piura", 241);
mapa.addArista("Cajamarca", "Piura", 448);
mapa.addArista("Cajamarca", "Pucallpa", 940);
mapa.addArista("Ica", "Arequipa", 718);
mapa.addArista("Huancayo", "Pucallpa", 730);
mapa.addArista("Huancayo", "Cusco", 736);
mapa.addArista("Arequipa", "Cusco", 521);
mapa.addArista("Arequipa", "Tacna", 319);
mapa.addArista("Arequipa", "Puno", 297);
mapa.addArista("Cusco", "Puno", 388);
```

```

mapa.addArista("Cusco",      "Pucallpa",    833);
mapa.addArista("Puno",      "Tacna",        377);

```

Figura 1. Inicialización del grafo con las 12 ciudades y 17 conexiones del mapa peruano.

```

// Cola FIFO: cada elemento es un Nodo con ciudad, costo acumulado,
// camino recorrido y conjunto de visitados para evitar ciclos.
Queue<Nodo> frontera = new LinkedList<>();
frontera.add(new Nodo(origen, 0.0, caminoInicial, visitadosInicial));

while (!frontera.isEmpty()) {
    Nodo actual = frontera.poll(); // extrae el nodo más antiguo (FIFO)
    nodosExpandidos++;

    if (actual.ciudad.equals(destino)) {
        long tiempo = System.currentTimeMillis() - inicio;
        return new Resultado(actual.camino, actual.g,
                               nodosExpandidos, tiempo);
    }

    // Se expanden los vecinos no visitados y se agregan al final de la cola
    for (Par vecino : grafo.getVecinos(actual.ciudad)) {
        if (!actual.visitados.contains(vecino.ciudad)) {
            List<String> nuevoCamino = new ArrayList<>(actual.camino);
            nuevoCamino.add(vecino.ciudad);
            Set<String> nuevosVisitados = new HashSet<>(actual.visitados);
            nuevosVisitados.add(vecino.ciudad);
            frontera.add(new Nodo(vecino.ciudad,
                                   actual.g + vecino.distancia,
                                   nuevoCamino, nuevosVisitados));
        }
    }
}

```

Figura 2. Implementación del método BFS con cola FIFO y detección de ciclos por camino.

```

// Cola de prioridad ordenada por  $f(n) = g(n) + h(n)$  ascendente.

```

```

// El nodo con menor f(n) se expande primero.
PriorityQueue<Nodo> frontera = new PriorityQueue<>(
    Comparator.comparingDouble(n -> n.f)
);
Set<String> cerrados = new HashSet<>(); // nodos ya expandidos

double h0 = Heuristica.distanciaLineaRecta(origen, destino);
frontera.add(new Nodo(origen, 0.0, h0, new ArrayList<>(List.of(origen))));

while (!frontera.isEmpty()) {
    Nodo actual = frontera.poll(); // nodo con menor f(n)
    nodosExpandidos++;

    if (actual.ciudad.equals(destino)) {
        long tiempo = System.currentTimeMillis() - inicio;
        return new Resultado(actual.camino, actual.g,
            nodosExpandidos, tiempo);
    }

    // Con heurística consistente, el primer camino encontrado
    // a cada nodo es el óptimo: no es necesario reabrir cerrados.
    if (cerrados.contains(actual.ciudad)) continue;
    cerrados.add(actual.ciudad);

    for (Par vecino : grafo.getVecinos(actual.ciudad)) {
        if (!cerrados.contains(vecino.ciudad)) {
            double nuevoG = actual.g + vecino.distancia;
            double h = Heuristica.distanciaLineaRecta(vecino.ciudad, destino);
            List<String> nuevoCamino = new ArrayList<>(actual.camino);
            nuevoCamino.add(vecino.ciudad);
            frontera.add(new Nodo(vecino.ciudad, nuevoG, h, nuevoCamino));
        }
    }
}
}

```

Figura 3. Implementación del algoritmo A con cola de prioridad por $f(n)$ y conjunto de nodos cerrados.*

```

// Conversión de coordenadas de grados a radianes

```

```

double lat1 = Math.toRadians(coord1[0]);
double lon1 = Math.toRadians(coord1[1]);
double lat2 = Math.toRadians(coord2[0]);
double lon2 = Math.toRadians(coord2[1]);

// Fórmula de Haversine: distancia sobre la superficie esférica
double dLat = lat2 - lat1;
double dLon = lon2 - lon1;
double a = Math.pow(Math.sin(dLat / 2), 2)
           + Math.cos(lat1) * Math.cos(lat2)
           * Math.pow(Math.sin(dLon / 2), 2);
// R = 6371 km (radio medio de la Tierra)
return 2 * 6371 * Math.asin(Math.sqrt(a));

```

Figura 4. Cálculo de la distancia en línea recta entre dos ciudades mediante la fórmula de Haversine.

5. Resultados y análisis comparativo

5.1. Casos de prueba

Se ejecutaron tres casos de prueba, seleccionados para representar rutas de diferente longitud y complejidad. En cada caso el usuario ingresa por consola el nombre exacto de la ciudad de origen y de destino; el programa valida el ingreso y repite la solicitud si la ciudad no existe en el mapa.

Caso 1: Lima → Cusco. Ruta de complejidad media con dos caminos competitivos: vía Ica–Arequipa (1542 km) y vía Huancayo (1034 km parcial). Es el caso principal para comparar los cuatro algoritmos. La salida de consola es la siguiente:

```

=====
BUSQUEDA DE RUTAS EN EL PERU - IA
Grupo 03 | UNMSM-FISI | 2026-I
=====

Ciudades disponibles:
Lima, Trujillo, Arequipa, Cusco, Piura, Tacna, Ica,
Puno, Chiclayo, Huancayo, Cajamarca, Pucallpa

Ingrese ciudad de origen : Lima
Ingrese ciudad de destino: Cusco

```

```
-----  
Buscando ruta de [Lima] a [Cusco]  
-----
```

```
--- BFS (no informada) ---
```

```
Ruta: Lima -> Huancayo -> Cusco
```

```
Costo: 1034 km | Nodos expandidos: 9 | Tiempo: 1 ms
```

```
--- DFS (no informada) ---
```

```
Ruta: Lima -> Trujillo -> Chiclayo -> Piura -> Cajamarca -> Pucallpa ->  
Huancayo -> Cusco
```

```
Costo: 3861 km | Nodos expandidos: 8 | Tiempo: 0 ms
```

```
--- Greedy Best-First (informada) ---
```

```
Ruta: Lima -> Huancayo -> Cusco
```

```
Costo: 1034 km | Nodos expandidos: 3 | Tiempo: 5 ms
```

```
--- A* (informada) ---
```

```
Ruta: Lima -> Huancayo -> Cusco
```

```
Costo: 1034 km | Nodos expandidos: 4 | Tiempo: 1 ms
```

Caso 2: Piura → Tacna. Ruta de extremo norte a extremo sur del país, que obliga a atravesar varias ciudades intermedias. Permite observar cómo BFS expande muchos nodos mientras A* se orienta directamente hacia el sur. La salida de consola muestra:

```
Ingrese ciudad de origen : Piura
```

```
Ingrese ciudad de destino: Tacna
```

```
--- BFS (no informada) ---
```

```
Ruta: Piura -> Cajamarca -> Pucallpa -> Cusco -> Arequipa -> Tacna
```

```
Costo: 3061 km | Nodos expandidos: 37 | Tiempo: 2 ms
```

```
--- DFS (no informada) ---
```

```
Ruta: Piura -> Chiclayo -> Trujillo -> Lima -> Ica -> Arequipa -> Cusco -> Puno  
-> Tacna
```

```
Costo: 3314 km | Nodos expandidos: 12 | Tiempo: 0 ms
```

```

--- Greedy Best-First (informada) ---
Ruta: Piura -> Cajamarca -> Pucallpa -> Cusco -> Arequipa -> Tacna
Costo: 3061 km | Nodos expandidos: 6 | Tiempo: 6 ms

--- A* (informada) ---
Ruta: Piura -> Chiclayo -> Trujillo -> Lima -> Ica -> Arequipa -> Tacna
Costo: 2347 km | Nodos expandidos: 10 | Tiempo: 1 ms

```

Caso 3: Cajamarca → Puno. Ruta entre dos ciudades sin conexión directa, con varias rutas alternativas de costo similar, lo que resulta exigente para la heurística:

```

Ingrese ciudad de origen : Cajamarca
Ingrese ciudad de destino: Puno

--- BFS (no informada) ---
Ruta: Cajamarca -> Pucallpa -> Cusco -> Puno
Costo: 2161 km | Nodos expandidos: 18 | Tiempo: 2 ms

--- DFS (no informada) ---
Ruta: Cajamarca -> Trujillo -> Lima -> Ica -> Arequipa -> Cusco -> Puno
Costo: 2782 km | Nodos expandidos: 9 | Tiempo: 0 ms

--- Greedy Best-First (informada) ---
Ruta: Cajamarca -> Pucallpa -> Cusco -> Puno
Costo: 2161 km | Nodos expandidos: 4 | Tiempo: 5 ms

--- A* (informada) ---
Ruta: Cajamarca -> Pucallpa -> Cusco -> Puno
Costo: 2161 km | Nodos expandidos: 11 | Tiempo: 1 ms

```

5.2. Tabla comparativa

Tabla 1. Comparación de algoritmos de búsqueda — Caso 1: Lima → Cusco

Algoritmo	Nodos expandidos	Costo (km)	Tiempo (ms)	Completo	Óptimo
BFS	9	1034	1	Sí	No
DFS	8	3861	0	Sí	No

Greedy	3	1034	5	No	No
A*	4	1034	1	Sí	Sí

Tabla 2. Comparación de algoritmos de búsqueda — Caso 2: Piura → Tacna

Algoritmo	Nodos expandidos	Costo (km)	Tiempo (ms)	Completo	Óptimo
BFS	37	3061	2	Sí	No
DFS	12	3314	0	Sí	No
Greedy	6	3061	6	No	No
A*	10	2347	1	Sí	Sí

Tabla 3. Comparación de algoritmos de búsqueda — Caso 3: Cajamarca → Puno

Algoritmo	Nodos expandidos	Costo (km)	Tiempo (ms)	Completo	Óptimo
BFS	18	2161	2	Sí	No
DFS	9	2782	0	Sí	No
Greedy	4	2161	5	No	No
A*	11	2161	1	Sí	Sí

5.3. Análisis de resultados

Los resultados obtenidos en los tres casos de prueba permiten extraer conclusiones concretas sobre el comportamiento de cada algoritmo.

BFS expande la mayor cantidad de nodos en los casos donde hay muchas rutas alternativas, como el Caso 2 con 37 nodos. Al explorar por niveles, garantiza encontrar primero la ruta con el menor número de saltos, no la de menor costo. En el Caso 1, la ruta con menor número de saltos (Lima–Huancayo–Cusco, 2 saltos) coincide con la de menor costo (1034 km), por lo que BFS obtiene el óptimo de forma accidental. En el Caso 2, sin embargo, la ruta con menor número de saltos (5 saltos vía Cajamarca–Pucallpa–Cusco–Arequipa, 3061 km) es más cara que la de menor costo (6 saltos vía Chiclayo–Lima–Ica–Arequipa, 2347 km), y BFS devuelve la

primera: esto evidencia de forma clara que BFS no optimiza costo en grafos ponderados (Russell & Norvig, 2022).

DFS es el que menos nodos expande en los tres casos (entre 8 y 12), pero produce sistemáticamente las rutas más costosas: 3861 km en el Caso 1, 3314 km en el Caso 2 y 2782 km en el Caso 3. Esto ocurre porque profundiza por la primera rama disponible sin considerar los pesos de las aristas, pudiendo recorrer caminos muy largos antes de encontrar la meta (Cormen et al., 2022). Su bajo conteo de nodos no refleja eficiencia, sino que encontró una solución rápida aunque subóptima.

Greedy Best-First expande el menor número de nodos en los tres casos (entre 3 y 6), orientado en cada paso hacia el nodo con menor distancia en línea recta al destino. En los Casos 1 y 3 su solución coincide con el óptimo, pero en el Caso 2 devuelve 3061 km en lugar de los 2347 km de A^* . Esto confirma el límite teórico de Greedy: ignorar el costo acumulado $g(n)$ puede llevarlo a elegir un camino geográficamente directo pero costoso en carretera, como ocurre al pasar por Pucallpa (Russell & Norvig, 2022).

A^* es el único algoritmo que garantiza la solución óptima en los tres casos: 1034 km, 2347 km y 2161 km respectivamente. Expande más nodos que Greedy pero menos que BFS, logrando el mejor equilibrio entre eficiencia y optimalidad gracias a la función $f(n) = g(n) + h(n)$ y a la admisibilidad de la heurística de Haversine (Russell & Norvig, 2022; Skiena, 2020). El Caso 2 es el ejemplo más ilustrativo: A^* es el único que encuentra la ruta real más corta (2347 km), mientras BFS y Greedy quedan en 3061 km.

En síntesis, A^* es el algoritmo recomendado cuando se requiere la solución de menor costo con una heurística admisible disponible. Greedy es la mejor opción cuando se prioriza velocidad y se acepta un posible margen de error. BFS es adecuado solo cuando se desea minimizar el número de pasos, no el costo. DFS no es recomendable en grafos ponderados donde el objetivo es encontrar la ruta más corta.

6. Conclusiones

El desarrollo de este trabajo permitió comprender de forma práctica las diferencias teóricas y empíricas entre los algoritmos de búsqueda no informada e informada aplicados a un problema real de navegación en mapa.

La primera conclusión es que la heurística reduce drásticamente el número de nodos expandidos: Greedy expandió entre 3 y 6 nodos por caso, frente a los 9–37 de BFS. Esto confirma que una heurística admisible y consistente, como la distancia de Haversine, es un recurso valioso para reducir el espacio de búsqueda sin sacrificar corrección en el caso de A* (Russell & Norvig, 2022).

La segunda conclusión es que el Caso 2 (Piura → Tacna) ilustra con claridad la diferencia fundamental entre los algoritmos: BFS encontró la ruta de menor número de saltos (3 061 km, 5 saltos), Greedy siguió la dirección geográfica más inmediata y llegó al mismo resultado no óptimo, mientras que A* identificó correctamente la ruta de menor costo (2 347 km, 6 saltos). Que una ruta con más pasos sea más barata es un escenario frecuente en grafos ponderados, y solo A* lo resuelve con garantía (Russell & Norvig, 2022).

La tercera conclusión es que DFS expandió el menor número de nodos en todos los casos, pero produjo siempre las rutas más costosas, con diferencias de hasta el 273% respecto al óptimo en el Caso 1 (3 861 km frente a 1 034 km). Esto confirma que un bajo conteo de nodos no implica eficiencia cuando el resultado es una solución de mala calidad (Cormen et al., 2022).

Finalmente, el ejercicio de formalizar el problema como un espacio de estados y comparar los cuatro algoritmos sobre datos reales ejecutados en el programa reforzó los fundamentos teóricos expuestos en las secciones 3.1 a 3.5. La ejecución real evidenció comportamientos que los análisis teóricos predicen pero que solo se comprenden plenamente al observarlos en la práctica (Skiena, 2020).

7. Anexo de prompts

Nº	Objetivo	Herramienta	Texto del prompt	Resultado / Uso	Validación / Ajuste
P-01	Generar las conexiones y distancias entre 12 ciudades peruanas con distancias realistas en km.	Gemini 3.5 Thinking	"Dame una tabla con distancias aproximadas en km por carretera entre las siguientes ciudades peruanas: Lima, Trujillo, Arequipa, Cusco, Piura, Tacna, Ica, Puno, Chiclayo, Huancayo, Cajamarca y Pucallpa. Solo incluye conexiones directas por carretera principal. El grafo debe ser conexo."	Se obtuvo una lista de 17 aristas con distancias; se verificaron algunas contra fuentes oficiales.	El equipo contrastó distancias con Google Maps y ajustó valores que diferían en más del 10%.
P-02	Obtener las coordenadas geográficas (lat/lon) de las 12 ciudades para la heurística de Haversine.	Gemini 3.5 Thinking	"Dame las coordenadas geográficas en grados decimales (latitud y longitud) de estas ciudades peruanas: Lima, Trujillo, Arequipa, Cusco, Piura, Tacna, Ica, Puno, Chiclayo, Huancayo, Cajamarca, Pucallpa."	Se obtuvieron los 12 pares de coordenadas; se usaron directamente en la clase Heuristica.java.	Se verificaron contra Wikipedia y el IGN Perú para asegurar precisión.
P-03	Revisar la estructura integral del código Java completo (los 10 archivos del proyecto) para detectar errores lógicos, problemas de interconexión entre clases y fallos en la implementación de los algoritmos (especialmente A*).	Claude Sonnet	"Revisa el código fuente completo de este proyecto Java modular para la búsqueda de rutas en Perú (adjunto las 10 clases, incluyendo la estructura del Grafo, Nodos, Resultados y los algoritmos BFS, DFS, Voraz y A*). Identifica errores lógicos globales, problemas de acoplamiento, fallos en la actualización de costos g(n) y el manejo de conjuntos abiertos/cerrados en A*." + [código completo adjunto]	Se realizó una auditoría completa del sistema: se identificaron fallos en la integración de la lista de adyacencia del Grafo y dos problemas críticos en A* (falta de verificación del conjunto cerrado y cálculo incorrecto de f(n)). El equipo corrigió la	El equipo compiló el proyecto completo corregido, ejecutó la clase principal con los casos de prueba viales y verificó que todos los algoritmos (incluido A*) funcionaran de manera óptima y coordinada sin

N°	Objetivo	Herramienta	Texto del prompt	Resultado / Uso	Validación / Ajuste
				arquitectura modular completa.	errores de dependencias.

8. Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.

Instituto Geográfico Nacional del Perú. (2024). *Coordenadas geográficas de capitales de departamento*. IGN. <https://www.igm.gob.pe>

Oracle Corporation. (2024). *Java SE 17 Documentation — Collections Framework*. Oracle. <https://docs.oracle.com/en/java/javase/17/>

Russell, S., & Norvig, P. (2022). *Artificial Intelligence: A Modern Approach* (4.^a ed.). Pearson.

Skiena, S. S. (2020). *The Algorithm Design Manual* (3.^a ed.). Springer.